Knowing When You're Done: Termination Detection for Recurrent Latent Reasoners under Depth Shift

Md Fahim Sarker
(affiliation and e-mail)

Abstract

Recurrent latent reasoners can solve harder problems by iterating for longer, but the extra compute only helps if the model also knows when to stop. We study this termination problem in a setting where the answer to "is the computation finished?" is known exactly. For local recurrent models, a light-cone argument assigns every input x a lower bound $\mathrm{LB}(x)$ on the first step at which its answer can be certified; the bound grows with depth rather than length, and a local, size-invariant automaton attains it at every length. We introduce DepthShift, a benchmark with exact per-instance depth and splits that vary depth and length independently, and evaluate fifteen stopping rules on one backbone family. Jointly trained pooled gates (ACT, PonderNet, TRM-style heads) leak length: on deeper addition inputs ACT is 99.8% accurate while halting before $\mathrm{LB}(x)$ through a global shortcut, yet only 35.9% once inputs are also longer. Convergence rules stop before the bound on most deep inputs and are still correct, so premature stopping and premature error must be measured separately. On addition, per-position confidence halts at exactly $\mathrm{LB}(x)$ on every split, and the local completion heads we propose match it without improving on it; on segmented parity the same heads learn the training clock instead. Termination detection beyond the training depth appears to be limited less by the readout than by training data that never show an unfinished computation at a late step. Code, frozen splits and all runs are released.

1 Introduction

A recurrent or looped model applies the same block again and again, and every extra iteration buys more computation on the same input. This lets small networks extrapolate to instances harder than any seen in training, provided they are allowed to think for longer [1–4]. The promise is adaptive compute, with few steps on easy inputs and many on hard ones, and keeping it requires a stopping rule. The rules in common use, learned halting gates [5, 6], confidence thresholds and convergence tests [3, 7], are all fitted on the difficulty seen in training. Yet the inputs that matter most are the ones harder than anything in the training set, and on them the rule must decide that the computation is finished from information it was never rewarded for reading.

We study this decision in a setting where the right answer is known. In a *local* recurrent reasoner, one step moves information by at most r positions, so after t steps position i depends only on the input within radius rt, its light cone. If that window does not yet determine the target, some other input agrees on the window but has a different answer, and the model must be wrong on one of the two. Every input therefore has a lower bound $\mathrm{LB}(x) = \lceil d^*(x)/r \rceil$ on the first step at which its answer can be certified (Theorem 1), where

Preprint.

the *depth* $d^*(x)$ is the largest radius any position needs. Halting earlier returns an answer that the model's own state does not determine, correct only by luck of the data. Because the bound depends on depth and not on length, it separates two shifts that are usually confounded, longer inputs and deeper ones. For the tasks we study, a constant-size local automaton halts at exactly $\mathrm{LB}(x)$ at every length (Proposition 2), so the bound is attainable and we can ask which stopping rules attain it.

Our contributions are threefold. We prove the light-cone bound, which turns "halting too early" into a checkable per-instance quantity, derive its consequences for local commit-on-done rules and for rules that see only the length, and give closed-form depth oracles and a matching automaton for addition and segmented parity, each with an executable test (Section 2). We introduce DepthShift, whose frozen splits vary depth and length independently up to 12.8× the training depth (Section 3). And we evaluate fifteen stopping rules and four references on one backbone family, from post-hoc confidence and convergence tests through ACT, PonderNet and pooled TRM-style and v1 heads to a family of local *completion heads* that we introduce, scoring each by accuracy, steps, and how often it stops or errs before $\mathrm{LB}(x)$ (Sections 4 and 5).

The results tell a consistent story. Jointly trained pooled gates leak length: on addition, ACT is 99.8% accurate on deeper inputs with fewer steps than the bound allows, because it reads a global shortcut, and falls to 35.9% once inputs are both deeper and longer. Premature stopping and premature error are different things: convergence rules halt before the bound on most deep addition inputs and are still right, because a travelling carry keeps the hidden state moving until it lands. When pending positions are genuinely ambiguous, as in addition, per-position confidence is already a certificate; max-softmax halts at exactly $\mathrm{LB}(x)$ on every split, and our completion heads match it without improving on it. When they are not, as in parity, the completion heads learn the training clock, while pooled gates extrapolate in depth but fall below 1% once inputs are also longer. Finally, jointly trained completion heads leave the backbone's solution intact, whereas gates that reweight the task loss degrade it. The difficulty of termination detection thus lies less in the readout than in training data that never show a computation still unfinished at a late step.

2 When can an answer be certified?

We first make precise what it means for a recurrent model to be "done". The argument is a light-cone argument: when information travels at bounded speed, a position cannot know about input it has not yet had time to see.

Local recurrent reasoners. An input is a sequence $x = (x_0, \dots, x_{P-1}) \in \Sigma^P$, and the task assigns a target $y_i(x) \in \Gamma$ to every position. We pad the input with a symbol $\perp \notin \Sigma$, writing $\tilde{x}_j = x_j$ for $0 \le j < P$ and $\tilde{x}_j = \perp$ otherwise. A *local recurrent reasoner* with growth rate $r \in \mathbb{N}$ keeps one latent vector $h_i^{(t)} \in \mathbb{R}^C$ per position and step and computes

$$\begin{aligned} e_i &= E(\tilde{x}_i), & h_i^{(0)} &= H(e_i), \\ h_i^{(t+1)} &= U\big(h_{i-r:i+r}^{(t)},\ e_{i-r:i+r}\big), & \hat{y}_i^{(t)} &= \arg\max_{\gamma \in \Gamma} D_\gamma\big(h_i^{(t)}\big), \end{aligned} \tag{1}$$

where the embedding E, the initial map H and the decoder D act pointwise, and the update U is shared across positions and steps. Our backbone (Section 4) is an instance with r kernel-3 convolutions per step. The information available to position i is described by the window of radius ρ around it, padding included:

$$W_i^\rho(x) = \big(\tilde{x}_{i-\rho}, \dots, \tilde{x}_{i+\rho}\big) \in (\Sigma \cup \{\perp\})^{2\rho+1}. \tag{2}$$

Lemma 1 (Light cone)**.** *For every $t \ge 0$ there is a function Φ_t, independent of i, with $h_i^{(t)} = \Phi_t\big(W_i^{rt}(x)\big)$. Consequently, if $W_i^{rt}(x) = W_{i'}^{rt}(x')$, then $\hat{y}_i^{(s)}(x) = \hat{y}_{i'}^{(s)}(x')$ for all $s \le t$.*

Per-instance depth. Since the window is all that a position can have seen, the natural measure of difficulty is how large the window must be before the answer is pinned down.

Definition 1 (Determining radius and depth)**.** Position i of x is *ρ-determined* if every pair (x', i') with $W^{\rho}_{i'}(x') = W^{\rho}_{i}(x)$ satisfies $y_{i'}(x') = y_i(x)$. The determining radius and the depth of the instance are

$$\rho_i(x) = \min\bigl\{\rho \in \mathbb{N} : i \text{ is } \rho\text{-determined in } x\bigr\}, \qquad d^*(x) = \max_{0 \le i < P} \rho_i(x). \tag{3}$$

The partner position i' is arbitrary, which matches what a translation-equivariant model can know: it learns where the input boundary is only once $\perp$ has entered its window.

Theorem 1 (Light-cone lower bound)**.** *Let f be a local recurrent reasoner with growth rate r, and let $t < \lceil \rho_i(x)/r \rceil$. Then there is a pair (x', i') with*

$$\hat{y}^{(t)}_{i'}(x') = \hat{y}^{(t)}_{i}(x) \qquad \textit{but} \qquad y_{i'}(x') \neq y_i(x),$$

so at step t the model is wrong at position i of x or at position i' of x'. In particular, no local reasoner is correct on every input at a step earlier than

$$\mathrm{LB}(x) = \left\lceil \frac{d^*(x)}{r} \right\rceil. \tag{4}$$

The theorem constrains every *stopping rule*. If a rule halts input x at step $\tau(x)$ with $\tau(x) < \mathrm{LB}(x)$, it returns an answer that the information held by the model does not certify. Whether that answer happens to be wrong depends on which member of the pair the data contain. Two consequences make this concrete.

Corollary 1 (Commit-on-done local rules)**.** *Suppose each position raises a done flag that depends only on its own history $h^{(0)}_i, \dots, h^{(t)}_i$ and commits $\hat{y}^{(t)}_i$ when it does. If the system is correct on every input, then position i of x commits no earlier than $\lceil \rho_i(x)/r \rceil$, and the input halts no earlier than $\mathrm{LB}(x)$.*

Corollary 2 (Rules that see only the length)**.** *For addition of L-digit numbers, $d^* \in \{1, \dots, L+1\}$. A rule whose halting step $\tau(L)$ depends only on L is correct on every input of length L only if $\tau(L) \ge \lceil (L+1)/r \rceil$, in which case it wastes $\lceil (L+1)/r \rceil - \mathrm{LB}(x)$ steps on input x. For uniformly random digits $d^* = O(\log L)$ with high probability, so the waste is $\Theta(L)$ steps. Conversely, a length-only rule fitted on training depths, with $\tau(L) < \lceil (L+1)/r \rceil$, is wrong on one member of every flip pair with $d^* > r\,\tau(L)$.*

The bound is attainable with local, size-invariant signals. For our two exact tasks the determining radius has a closed form, and a local automaton reaches the bound at every input length. For addition we write digits least-significant first, with outputs at positions $0, \dots, L$ (position L holds the final carry), and call $j < L$ a *stop* if $a_j + b_j \neq 9$. A stop either generates a carry or absorbs one, so it is the only kind of position whose carry-out is known without looking further left. Let

$$s_i(x) = \max\bigl(\{-1\} \cup \{\, j < i : a_j + b_j \neq 9 \,\}\bigr) \tag{5}$$

be the nearest stop to the left of i, with -1 standing for the boundary.

Proposition 1 (Depth oracles)**.** *For addition, $\rho_i(x) = i - s_i(x)$ for every output position i. Hence $d^*(x)$ is the largest gap between consecutive elements of $\{-1\} \cup \{\text{stops}\} \cup \{L\}$, which equals one plus the longest run of digit pairs summing to 9. For segmented parity, $\rho_i(x) = i - s_i(x)$ for every non-delimiter position i, where $s_i(x)$ is now the previous delimiter (or -1), and $d^*(x)$ is the length of the longest run of non-delimiters.*

Proposition 2 (Achievability)**.** *For addition and segmented parity there is a local reasoner with $r = 1$ and a commit-on-done local flag such that, for every input of every length, every committed answer is correct and the input halts at exactly $\mathrm{LB}(x) = d^*(x)$ steps.*

The automaton behind Proposition 2 keeps a single bit of *pending* state per position, recording whether its carry-in (or its segment prefix parity) is known yet, and resolves it as soon as a resolved neighbour passes its value along. The done flag is a pointwise function of this state, and the same rule works at every length. This is the inductive bias our completion heads aim for. A pooled gate or a global confidence statistic, by contrast, has no structural reason to wait for the slowest pending chain, and Section 5 tests whether learned signals behave like the automaton beyond the training depth. Proofs are in Section B, and every result is checked in `tests/test_theory.py` (flip tests, pair constructions, brute-force oracles, and the automata on lengths up to 512).

Table 1: DepthShift splits for addition (T1) and segmented parity (T2). Every OOD split is evaluated once, at the end; thresholds are selected on ID-val.

Split	Length	Depth d^*	Instances	Purpose
Train (sampled online)	4–20	1–10	–	training
ID-val / ID-test	4–20	1–10	2000 / 2000	thresholds / test
OOD-val	24–32	12–16	1000	backbone tuning only
OOD-depth	4–20[a]	11–20	2000	deeper, same length
OOD-length	64–256	1–10	1000	longer, same depth
OOD-both	64–256	32–128	1000	deeper and longer

[a] 10–20 for addition and 11–20 for parity, the shortest lengths that admit the depth.

3 The DepthShift benchmark

DepthShift pairs tasks that have an exact per-instance depth oracle with a split design that separates *longer* inputs from *deeper* ones (Figure 2 in Section D). All generators run on the GPU from fixed seeds. Test sets are frozen and stored with SHA-256 hashes, and the training sampler discards any instance whose hash matches an in-distribution validation or test instance.

T1: addition. Each instance consists of two L-digit numbers a and b, written least-significant digit first and encoded as one token $10a_i + b_i$ per position, plus one extra position for the final carry. By Proposition 1, the depth is one plus the longest run of digit pairs that sum to 9, because a carry generated at a stop must ripple through the whole run before the positions behind it are determined. Uniformly random digits make long runs exponentially rare (each pair sums to 9 with probability $1/10$), so we control d^* directly. The generator inserts exactly one run of length $d^* - 1$ between two stops, draws background runs at a random density, and caps them below $d^* - 1$. Stops are drawn uniformly from the 90 pairs with $a + b \neq 9$, which means a run receives an incoming carry with probability $1/2$.

T2: segmented parity. The input is a bit string interleaved with delimiter tokens, and the target at each position is the parity of the current segment up to that position. The depth is the length of the longest segment. Parity differs from addition in a way that turns out to matter: as information arrives from the left, the intermediate predictions keep flipping, so the per-step outputs do not settle early.

T3: mazes. We also generate $n \times n$ mazes with a randomized depth-first search; the input channels mark open cells, the start and the goal, and the target is the shortest path. We record the optimal path length and the number of dead-end-filling iterations a local rule needs to isolate the path as depth proxies. The generator and splits are released, but within our compute budget no maze backbone passed Gate G1 (Section A), so all experiments below use T1 and T2.

Splits. Table 1 summarises the splits for T1 and T2. In-distribution (ID) instances have lengths 4–20 and depths 1–10. OOD-depth keeps these lengths and raises the depth to 11–20; OOD-length raises the length to 64–256 at ID depths; OOD-both combines lengths 64–256 with depths 32–128, up to 12.8× the training maximum. A separate OOD-validation split (lengths 24–32, depths 12–16) is used only to tune the backbone, and the thresholds of every stopping rule are chosen on ID-validation. For mazes we train on 9×9 grids and test on 13×13 up to 33×33.

4 Backbone and completion heads

Backbone. Our backbone, GLWM-2, instantiates Equation (1) with a pointwise embedding E, the initial state $H = \mathrm{LN}$, a pointwise two-layer MLP decoder D, and the shared step

$$h^{(t+1)} = \mathrm{LN}\Big(h^{(t)} + W_{\mathrm{out}}\,\mathrm{GELU}\big(\mathrm{Conv}_3^{(r)} \circ \cdots \circ \mathrm{GELU} \circ \mathrm{Conv}_3^{(1)}\,[h^{(t)}; e]\big)\Big). \tag{6}$$

Each step applies r kernel-3 convolutions to the current state concatenated with the input embedding (input re-injection, or "recall"), adds a residual connection, and normalises over channels at every position. Each convolution widens the receptive field by one position on either side, so the growth rate is exactly r, which we verify with the flip test. We use width $C = 128$ and $r = 1$. The backbone is trained with the per-step cross-entropy averaged uniformly over $T_{\text{train}} = 20$ steps (the "fixed prior" objective) or with the progressive loss of Bansal et al. [2]; a pilot (Section 5) selects the objective for each task.

Completion heads. A completion head reads the latent state of each position and predicts whether that position is *done*, meaning that its prediction is correct now and will remain correct at every later step. The target comes from the model's own trajectory up to a horizon T:

$$z_i^{(t)} = \prod_{s=t}^{T} \mathbb{1}\big[\hat{y}_i^{(s)} = y_i\big] = \min_{t \le s \le T} \mathbb{1}\big[\hat{y}_i^{(s)} = y_i\big], \tag{7}$$

that is, a reversed cumulative minimum of per-position correctness, and the head is trained with binary cross-entropy against $z_i^{(t)}$. We study three variants. The *post-hoc* head is a pointwise MLP $p_i^{(t)} = \sigma\big(g(h_i^{(t)})\big)$ fitted after the backbone has been frozen, on rollouts of length $2T_{\text{train}}$. The *joint* head has the same form but is trained alongside the backbone with an auxiliary binary cross-entropy of weight 1. Unlike ACT and PonderNet, it never reweights the per-step task loss: the backbone receives the same task gradient at every step, so predicting completion is a separate problem rather than a halting distribution laid over the task loss. Finally, the *propagating* head also reads its neighbours' previous done probabilities,

$$p_i^{(t)} = \sigma\Big(g\big(h_i^{(t)}\big) + \mathrm{Conv}_3\big[p^{(t-1)}, \mathbf{1}\big]_i\Big), \tag{8}$$

which mirrors the pending-bit automaton of Proposition 2: a position can learn to wait until its left neighbour reports that it is done.

Halting rule. An instance halts at the first step at which every valid position is done with probability above a threshold θ, or at the budget $T_{\max}$:

$$\tau_\theta(x) = \min\Big(\big\{\, t \ge 1 : \min_{0 \le i < P} p_i^{(t)}(x) > \theta \,\big\} \cup \{T_{\max}\}\Big), \tag{9}$$

and returns $\hat{y}^{(\tau_\theta(x))}(x)$ (Algorithm 1 in Section C). We select θ on ID-val as the cheapest threshold whose accuracy is within 0.5 points of the best. Because the rule takes a minimum over positions, its cost is set by the slowest position rather than by the input length.

Baselines. Every baseline uses the same backbone and the same selection protocol. As references we report a fixed step count T selected on ID-val, the oracle lower bound $\mathrm{LB}(x)$, the any-step oracle (the first step with the right answer) and the stable oracle (the first step from which the answer stays right). Writing $\pi_i^{(t)} = \mathrm{softmax}\, D(h_i^{(t)})$ for the predictive distribution at position i, four post-hoc rules halt once a scalar statistic crosses a threshold:

$$\begin{aligned} S_{\text{msp}}^{(t)} &= \min_i \max_\gamma \pi_{i\gamma}^{(t)}, & S_{\text{ent}}^{(t)} &= \max_i \mathrm{H}\big(\pi_i^{(t)}\big), \\ S_{\text{KL}}^{(t)} &= \max_i \mathrm{KL}\big(\pi_i^{(t)} \,\big\|\, \pi_i^{(t-1)}\big), & S_{\Delta h}^{(t)} &= \frac{\|h^{(t)} - h^{(t-1)}\|_F}{\|h^{(t)}\|_F}, \end{aligned} \tag{10}$$

where the first halts when it rises above its threshold and the other three when they fall below it. Note that max-softmax and entropy are per-position statistics combined by a worst case, whereas the hidden-state change pools over the whole input. We further include k-step prediction stability [7], maximum confidence over steps [8], and a pooled correctness head in the style of TRM [4] fitted post hoc. The jointly trained rules are ACT [5], PonderNet with its KL term towards a geometric prior [6], a joint pooled correctness head, and the pooled gate of our earlier GLWM model (v1) with its initialisation fixed.

Table 2: Addition (T1): accuracy (%) and mean steps of every stopping rule. Rows: references, post-hoc rules, jointly trained pooled gates, completion heads. Mean of two seeds; brackets are bootstrap intervals over the seeds.

	id-test		ood-depth		ood-length		ood-both	
	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps
Fixed T	100.0 [100.0, 100.0]	10.0	70.9 [69.5, 72.3]	10.0	100.0 [100.0, 100.0]	10.0	49.5 [49.5, 49.5]	10.0
Oracle (stable)	100.0 [100.0, 100.0]	2.9	100.0 [100.0, 100.0]	4.9	100.0 [100.0, 100.0]	4.4	99.6 [99.6, 99.7]	40.3
Max-softmax	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.8 [99.8, 99.8]	75.6
Entropy	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.8]	75.4
k-step stability	99.9 [99.9, 100.0]	3.8	99.9 [99.8, 100.0]	5.2	100.0 [100.0, 100.0]	5.4	99.8 [99.7, 99.9]	40.5
KL between steps	100.0 [100.0, 100.0]	4.0	99.8 [99.7, 100.0]	5.3	100.0 [100.0, 100.0]	5.6	99.8 [99.7, 99.9]	40.6
Hidden-state change	100.0 [100.0, 100.0]	6.2	100.0 [100.0, 100.0]	14.1	69.5 [69.4, 69.6]	4.3	42.7 [42.6, 42.8]	4.2
Max confidence (Fan et al.)	100.0 [100.0, 100.0]	26.0	100.0 [100.0, 100.0]	36.0	100.0 [100.0, 100.0]	26.0	99.8 [99.8, 99.9]	144.0
Correctness head (post hoc)	99.6 [99.4, 99.8]	3.6	99.5 [99.1, 99.9]	5.4	52.4 [51.9, 52.8]	3.3	50.0 [49.6, 50.4]	50.0
ACT	99.9 [99.9, 99.9]	2.6	99.8 [99.8, 99.8]	2.4	43.2 [42.7, 43.7]	2.8	35.9 [35.6, 36.3]	3.0
PonderNet	100.0 [100.0, 100.0]	5.0	99.5 [99.2, 99.9]	6.5	81.3 [81.3, 81.4]	5.2	47.6 [47.5, 47.7]	5.5
Correctness head (joint)	100.0 [100.0, 100.0]	2.9	100.0 [100.0, 100.0]	4.2	48.9 [45.3, 52.5]	3.0	31.9 [24.3, 39.5]	15.1
v1 GLWM gate	99.9 [99.9, 99.9]	2.4	99.8 [99.8, 99.8]	2.1	49.6 [46.3, 52.9]	4.4	69.7 [62.5, 76.8]	4.0
Completion head (post hoc)	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.7]	75.4
Propagating head (post hoc)	100.0 [100.0, 100.0]	5.1	99.8 [99.8, 99.9]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.7]	75.4
Completion head (joint)	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.0 [98.4, 99.7]	75.4
Propagating head (joint)	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	100.0 [100.0, 100.0]	75.4

5 Experiments

Protocol. All rules read the same frozen backbone checkpoints, except the jointly trained methods, which train their own. Thresholds are selected on ID-val (Section 4), and each OOD split is evaluated exactly once. For a test set $\{(x_n, y_n)\}_{n=1}^{N}$ and a rule that halts input x_n at step τ_n, we report exact-match accuracy over all output positions, the mean number of steps, and two error modes measured against the per-instance bound of Equation (4):

$$\mathrm{PS} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{1}\big[\tau_n < \mathrm{LB}(x_n)\big], \qquad \mathrm{PE} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{1}\big[\tau_n < \mathrm{LB}(x_n)\big]\, \mathbb{1}\big[\hat{y}^{(\tau_n)}(x_n) \neq y_n\big]. \quad (11)$$

The *premature-stop rate* PS counts uncertified answers, and the *premature-error rate* $\mathrm{PE} \leq \mathrm{PS}$ counts those that are also wrong. The brackets in the accuracy tables are percentile-bootstrap intervals over the two seeds, so they span roughly the seed range; premature rates use a bootstrap over test instances pooled across seeds. With two seeds a Wilcoxon signed-rank test cannot reach significance (the smallest attainable p is 0.5), so the pre-registered Holm-corrected tests are reported in `results/paper_numbers.json` but no claim rests on them; the effects we discuss are tens of points, far above seed-to-seed variation. The objective pilot was a tie, so both backbones use the fixed prior (`PREREGISTRATION.md`).

Backbones (Gate G1). We first check that the backbones can solve the deep splits at all, by stopping them with the stable oracle. The addition backbone is then correct on 99.6% of OOD-both instances, whose depths reach 12.8× the training maximum (Figure 5), so it passes G1. The parity backbone reaches 100% on OOD-depth and OOD-length but only 85% on OOD-both, and its per-step accuracy decays after the peak (Figure 4). It fails G1, and on parity we therefore measure how stopping rules behave on an imperfect backbone. Adaptive halting is clearly needed: a fixed step count chosen on ID-val is correct on only 71% (addition) and 31% (parity) of OOD-depth instances.

Pooled gates leak length (H1). The jointly trained pooled gates look excellent in distribution, and even on OOD-depth, but collapse as soon as the inputs grow longer (Table 2). On addition, ACT with the tuned ponder weight $\tau_{\mathrm{ACT}} = 0.01$ is correct on 99.8% of OOD-depth instances in only 2.4 steps, far below the light-cone bound, yet on just 43.2% of OOD-length and 35.9% of OOD-both instances. Inspecting its decisions reveals a shortcut. ACT halts at step 1 when no carry can enter the long run of nines and at step 5 otherwise, and the backbone's default output for a pending position switches from "no carry" to "carry" at around step 4. A pooled gate can read this global bit reliably when the input contains a single long run, but the bit becomes useless once there are many runs. PonderNet, the v1 gate and both TRM-style pooled correctness heads fail in the same way, reaching 47.6%, 69.7%, 50.0% (post hoc) and 31.9% (joint) on OOD-both. Their step counts are nearly flat

Table 3: Segmented parity (T2), same layout as Table 2. The backbone fails G1 on OOD-both (stable oracle 85%).

	id-test Acc. (%)	Steps	ood-depth Acc. (%)	Steps	ood-length Acc. (%)	Steps	ood-both Acc. (%)	Steps
Fixed T	100.0 [100.0, 100.0]	9.0	30.9 [30.0, 31.8]	9.0	100.0 [100.0, 100.0]	9.0	0.0 [0.0, 0.0]	9.0
Oracle (stable)	100.0 [100.0, 100.0]	3.5	100.0 [100.0, 100.0]	11.8	100.0 [100.0, 100.0]	4.5	85.4 [79.8, 90.9]	90.1
Max-softmax	100.0 [100.0, 100.0]	4.9	51.0 [50.3, 51.8]	10.3	100.0 [100.0, 100.0]	5.4	0.8 [0.4, 1.2]	37.4
Entropy	100.0 [100.0, 100.0]	4.9	35.2 [33.7, 36.8]	9.4	100.0 [100.0, 100.0]	5.4	0.1 [0.0, 0.1]	17.9
k-step stability	100.0 [100.0, 100.0]	4.3	51.0 [45.0, 57.0]	10.7	100.0 [100.0, 100.0]	5.4	4.5 [2.0, 7.0]	54.1
KL between steps	100.0 [100.0, 100.0]	5.9	94.0 [90.6, 97.5]	13.0	100.0 [100.0, 100.0]	6.4	56.6 [35.8, 77.5]	83.9
Hidden-state change	100.0 [100.0, 100.0]	6.7	63.2 [59.2, 67.2]	11.0	98.6 [98.6, 98.7]	6.8	0.0 [0.0, 0.0]	13.2
Max confidence (Fan et al.)	100.0 [100.0, 100.0]	26.0	100.0 [100.0, 100.0]	36.0	100.0 [100.0, 100.0]	26.0	85.2 [79.6, 90.7]	144.0
Correctness head (post hoc)	99.8 [99.7, 100.0]	4.6	32.2 [31.7, 32.6]	9.1	86.1 [84.1, 88.2]	4.9	0.0 [0.0, 0.0]	8.5
ACT	100.0 [100.0, 100.0]	3.5	98.9 [97.9, 100.0]	11.4	86.2 [86.2, 86.2]	4.3	0.7 [0.2, 1.2]	52.4
PonderNet	100.0 [100.0, 100.0]	5.5	96.7 [94.5, 98.8]	13.1	100.0 [100.0, 100.0]	6.4	0.1 [0.0, 0.1]	58.9
Correctness head (joint)	100.0 [100.0, 100.0]	3.5	90.9 [88.6, 93.2]	11.3	81.0 [78.1, 84.0]	4.3	0.1 [0.1, 0.2]	27.0
v1 GLWM gate	3.0 [2.9, 3.0]	26.0	0.0 [0.0, 0.0]	36.0	0.0 [0.0, 0.0]	26.0	0.0 [0.0, 0.0]	144.0
Completion head (post hoc)	99.8 [99.6, 100.0]	4.9	30.6 [29.9, 31.2]	9.0	100.0 [99.9, 100.0]	5.4	0.0 [0.0, 0.0]	9.0
Propagating head (post hoc)	100.0 [100.0, 100.0]	4.9	30.6 [30.0, 31.1]	9.0	100.0 [100.0, 100.0]	5.4	0.0 [0.0, 0.0]	9.0
Completion head (joint)	99.8 [99.7, 100.0]	4.9	32.2 [31.9, 32.6]	9.0	99.8 [99.5, 100.0]	5.4	0.0 [0.0, 0.0]	8.9
Propagating head (joint)	100.0 [100.0, 100.0]	4.9	34.2 [33.1, 35.3]	9.0	100.0 [100.0, 100.0]	5.4	0.0 [0.0, 0.0]	9.0

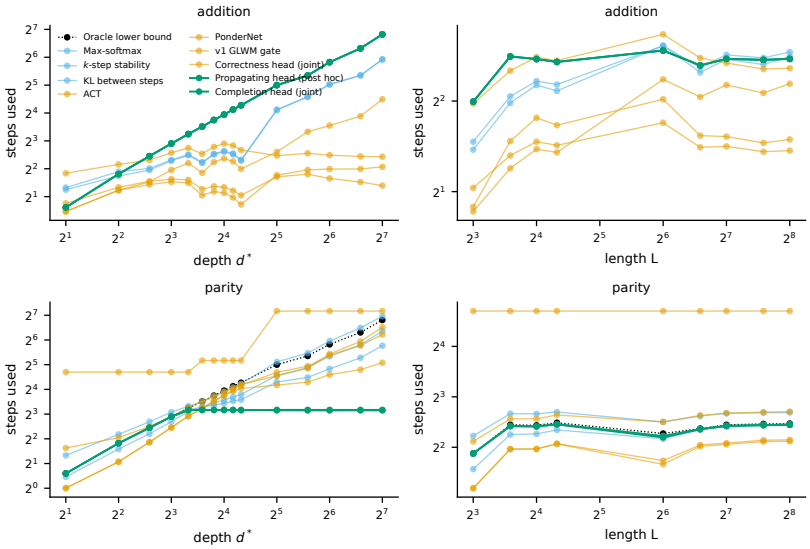


Figure 1: Steps used against depth (left) and length (right). The light-cone bound (dotted) grows with depth only. Per-position rules on addition track it; pooled gates stay flat.

in depth (Figure 1): out of distribution they spend what they spent in training. Parity is more forgiving in depth, where ACT and PonderNet do scale their steps and reach 98.9% and 96.7% on OOD-depth, but both fall below 1% on OOD-both. The v1 gate did not train on parity at all (3% ID accuracy): it never halts, and its halting-weighted loss degrades the backbone.

Premature stopping is not premature error (H2). The convergence rules, k-step stability and the KL between steps, halt before LB(x) on 75% of OOD-depth and 50% of OOD-both addition instances (Table 4), and yet they remain 99.8% accurate. Pooled over the deep splits, their premature-error rate is 0.1%, compared with 18–23% for ACT, PonderNet and the joint pooled head. This is precisely the gap that Theorem 1 leaves open: the answers are uncertified, but the test data do not contain the flip partner. The backbone's default guess for a pending digit is usually right, and while a carry wave is still travelling the hidden state keeps changing, so convergence is a good proxy for completion. Parity shows the effect in its sharpest form: one step before the bound a position has seen every bit of its segment, only the closing delimiter lies outside its window, and the natural guess that no further bits follow is correct on the actual input. This is why the stable oracle and ACT can

Table 4: Premature-stop rate PS (%), the fraction of instances halted before the light-cone bound LB(x). Such answers are uncertified (Theorem 1), though not necessarily wrong.

	addition		parity	
	OOD-depth	OOD-both	OOD-depth	OOD-both
Fixed T	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Max-softmax	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	90.8 [89.9, 91.7]	98.2 [97.5, 98.8]
Entropy	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	99.2 [98.9, 99.5]	100.0 [100.0, 100.0]
k-step stability	74.7 [73.3, 76.0]	49.7 [47.4, 51.9]	86.2 [85.2, 87.3]	85.5 [84.0, 87.1]
KL between steps	74.7 [73.3, 76.0]	49.7 [47.4, 51.9]	49.2 [47.7, 50.7]	28.5 [26.5, 30.6]
Hidden-state change	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	81.1 [79.9, 82.3]	100.0 [100.0, 100.0]
Max confidence (Fan et al.)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Correctness head (post hoc)	75.3 [73.9, 76.6]	99.6 [99.3, 99.9]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
ACT	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	97.6 [97.1, 98.0]	100.0 [100.0, 100.0]
PonderNet	75.1 [73.7, 76.5]	100.0 [100.0, 100.0]	30.3 [28.9, 31.7]	92.2 [91.1, 93.4]
Correctness head (joint)	74.7 [73.3, 76.0]	100.0 [100.0, 100.0]	92.6 [91.8, 93.4]	100.0 [100.0, 100.0]
v1 GLWM gate	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Completion head (post hoc)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Propagating head (post hoc)	0.7 [0.4, 0.9]	0.1 [0.0, 0.1]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Completion head (joint)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Propagating head (joint)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]

be accurate on parity OOD-depth while halting before the bound. The hidden-state-change rule, whose threshold is fitted on short inputs, halts after about four steps on OOD-both and is correct on 42.7% of instances. On parity, where outputs keep flipping as information arrives, confidence is actively misleading: max-softmax halts before the bound on 91% of OOD-depth instances and is correct on only 51%, and on OOD-both on 0.8%. Only the KL rule partly survives, with 94% on OOD-depth and 57% on OOD-both. Since both H1 and H2 hold, Gate G2 passes.

Per-position certificates on addition (H3). On addition, every per-position rule halts at the light-cone bound on every split, with no premature stops and at least 99% accuracy. Max-softmax, entropy and all four completion-head variants use 13.5 steps on OOD-depth and 75.4–75.6 steps on OOD-both, against 13.5 and 75.4 for the oracle LB(x). The theory explains why. A pending addition digit has two equally plausible outputs, carry or no carry, so a calibrated decoder is uncertain at exactly the positions that are not yet determined, and S_{msp} in Equation (10), a minimum over positions, becomes a local certificate in the sense of Corollary 1. A linear probe recovers the pending bit from the hidden state with AUC above 0.999 on every split (Table 7). Per-position confidence on addition therefore already implements the automaton of Proposition 2, and a learned completion head has nothing to add. Gate G3, which required a 10-point gain over the best baseline, fails on addition by construction.

Completion heads on parity learn the clock. On parity, all completion heads halt after about nine steps regardless of depth, close to the largest training depth. They are correct on 31–34% of OOD-depth and 0% of OOD-both instances, well below ACT and PonderNet on OOD-depth, and the joint and propagating variants behave no differently. The cause lies in the training data. During training every position is done by step 10, so the targets in Equation (7) make "late" and "done" indistinguishable, and the parity backbone's hidden state encodes elapsed time well enough for the head to exploit it. The propagating head of Equation (8) was designed to wait for its neighbours, but nothing in the data forces it to, since a neighbour is also always done by step 10. This is the failure of the pooled gates in another guise, with the shortcut moved from the input length to the step count. G3 fails on parity as well, and following the pre-registered pivot we read the results as a map of when simple signals suffice rather than as a win for a new halting method.

Decoupling (H4) and compute. Training a completion head jointly changes the backbone's step-averaged ID-val accuracy by less than 0.02 points on both tasks, so the auxiliary target leaves the task solution undisturbed. Gates that reweight the task loss do disturb it. Relative to the base model, ACT lowers this accuracy by 34 points and the v1 gate by 17 points on addition (86 points for the v1 gate on parity), because steps after the learned halting point receive little task gradient. In FLOPs, a completion head costs one pointwise

MLP per position and step, and the cost of every rule is dominated by the number of steps it takes (Figure 10). Wall-clock savings are smaller at our scale. With online early exit on 512 OOD-depth instances, max-softmax takes 19.8 ms against 23.8 ms for the full budget, while the completion heads take 23.8 and 28.3 ms (Table 7), because the host synchronisation and re-batching after every step cost about as much as the steps saved. Converting certified halting into latency will need fused halting checks or larger models.

Ablations. Removing input re-injection leaves OOD-depth intact but destroys OOD-both (stable oracle 0.6%). Halving the width to 64 lowers the stable oracle only to 95.2% but max-softmax to 48.0%: with less capacity, the decoder's confidence stops tracking the pending state. Two convolutions per step ($r = 2$) reach 100% on both (Table 7); each ablation uses a single seed. Trade-off curves and premature stopping by depth are in Figures 8 and 9.

6 Related work

Adaptive computation. ACT [5] and PonderNet [6] learn a halting distribution that weights the task loss; Universal Transformers apply ACT per token [9], and SACT halts per spatial location [10]. These gates are trained on the depths that occur in training, and we test them beyond those depths while separating their sensitivity to length from their sensitivity to depth.

Recurrent models that extrapolate. Neural GPUs learn long addition [11] but fail on long carry chains that random inputs rarely contain [12], an observation DepthShift turns into a controlled split. Recurrent networks extrapolate to harder instances by running more iterations [1, 2, 13], with the count chosen by the experimenter. Path independence makes convergence a usable signal [14], and test-time training can select compute per sample [15]. Looped Transformers [16–18], including those of Fan et al. [8], need ground-truth step counts in training and stop by oracle or maximum confidence at test time. AdRNNs [19] and ANIRA [20] halt later on harder inputs, but there harder also means larger, so depth and length are confounded. Implicit models solve for a fixed point instead [21].

Latent reasoning and recursive models. Recurrent-depth language models exit per token from the KL between steps [3], Ouro learns an exit gate [22], and Coconut reasons in continuous latents [23]. HRM [24] and TRM [4] train a pooled halting or correctness head, and HRM has been analysed by Ge et al. [25]. Recent diagnoses of halting in such models examine stochastic stopping schedules and the link between loop count and length [26], post-hoc confidence that matches learned gates [27], latent motion that drops after the first exact solution [28], and states that encode computational status but grow less reliable with depth [29]. We add a per-instance depth with a provable lower bound, splits that separate depth from length, and a local completion target.

Early exit, length generalisation and termination detection. Early-exit networks halt on confidence or patience [7, 30–32], CALM adds calibrated guarantees [33], and language models can predict their own correctness [34]. Early exit for reasoning models relies on answer convergence or dedicated predictors [35–38]. All of these signals are pooled over the whole answer, whereas ours is per position and tested beyond the training depth. Arithmetic and algorithmic tasks are standard probes of length generalisation [39–43], and shortcut solutions tie the depth a model needs to its architecture [44]. Finally, deciding from local information that a distributed computation has finished is a classic problem [45]; our completion heads learn such a detector for a neural computation, and Proposition 2 shows that a local, size-invariant one exists for our tasks. Table 8 in Section D compares the closest works property by property.

Limitations. All runs use one consumer GPU and a five-hour training budget, which allowed two seeds per method, single-seed ablations and a reduced tuning budget for PonderNet and the v1 gate. The exact theory covers one-dimensional tasks with closed-form depth, no maze backbone passed G1, and the parity backbone fails G1 on OOD-both; Section A discusses these points further.

Acknowledgments and Disclosure of Funding

Compute: one NVIDIA RTX 5070. GPU-hours are reported in Section C.

References

[1] Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can you learn an algorithm? generalizing from easy to hard problems with recurrent networks. arXiv:2106.04537, 2021.

[2] Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end algorithm synthesis with recurrent networks: Logical extrapolation without overthinking. arXiv:2202.05826, 2022.

[3] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. arXiv:2502.05171, 2025.

[4] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871, 2025.

[5] Alex Graves. Adaptive computation time for recurrent neural networks. arXiv:1603.08983, 2016.

[6] Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. arXiv:2107.05407, 2021.

[7] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. Bert loses patience: Fast and robust inference with early exit. arXiv:2006.04152, 2020.

[8] Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. arXiv:2409.15647, 2024.

[9] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2018.

[10] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry Vetrov, and Ruslan Salakhutdinov. Spatially adaptive computation time for residual networks. arXiv:1612.02297, 2016.

[11] Łukasz Kaiser and Ilya Sutskever. Neural gpus learn algorithms. arXiv:1511.08228, 2015.

[12] Eric Price, Wojciech Zaremba, and Ilya Sutskever. Extensions and limitations of the neural gpu. arXiv:1611.00736, 2016.

[13] Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Arpit Bansal, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. Datasets for studying generalization from easy to hard examples. arXiv:2108.06011, 2021.

[14] Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, Zico Kolter, and Roger Grosse. Path independent equilibrium models can better exploit test-time computation. arXiv:2211.09961, 2022.

[15] Hieu Tran Bao, Nguyen Cong Dat, Nguyen Duc Anh, and Hoang Thanh-Tung. Learning to stop overthinking at test time. arXiv:2502.10954, 2025.

[16] Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. arXiv:2301.13196, 2023.

[17] Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. arXiv:2311.12424, 2023.

[18] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. arXiv:2502.17416, 2025.

[19] Vijay Veerabadran, Srinivas Ravishankar, Yuan Tang, Ritik Raina, and Virginia R. de Sa. Adaptive recurrent vision performs zero-shot computation scaling to unseen difficulty levels. arXiv:2311.06964, 2023.

[20] Ibraheem Muhammad Moosa, Suhas Lohit, Ye Wang, Moitreya Chatterjee, and Wenpeng Yin. Understanding dynamic compute allocation in recurrent transformers. arXiv:2602.08864, 2026.

[21] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. arXiv:1909.01377, 2019.

[22] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models. arXiv:2510.25741, 2025.

[23] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. arXiv:2412.06769, 2024.

[24] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. arXiv:2506.21734, 2025.

[25] Renee Ge, Qianli Liao, and Tomaso Poggio. Hierarchical reasoning models: Perspectives and misconceptions. arXiv:2510.00355, 2025.

[26] Hsun-Yu Kuo, El Mahdi Chayti, Patrik Reizinger, Wieland Brendel, and Martin Jaggi. Stabilizing extrapolation in looped transformers via learned stochastic stopping. arXiv:2606.29983, 2026.

[27] Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive depth in looped transformers: Diagnosing learned halting gates and trajectory readouts. arXiv:2607.20519, 2026.

[28] Hare Krishna, Shubham Singh, Stephen Ebert, and Hao-Yu Sun. When recursive models finish computing. arXiv:2609.26487, 2026.

[29] Jia Liang, Xi Jin, and Liangming Pan. Beyond the training horizon: Mechanisms and limits of length generalization in looped transformers. arXiv:2609.33144, 2026.

[30] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. arXiv:1709.01686, 2017.

[31] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. arXiv:1910.10073, 2019.

[32] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference. arXiv:2004.12993, 2020.

[33] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Neural Information Processing Systems*, 2022.

[34] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine

Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. arXiv:2207.05221, 2022.

[35] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. arXiv:2504.15895, 2025.

[36] Xin Liu and Lu Wang. Answer convergence as a signal for early stopping in reasoning. arXiv:2506.02536, 2025.

[37] Alliot Nagle, Jakhongir Saydaliev, Dhia Garbaya, Michael Gastpar, Ashok Vardhan Makkuva, and Hyeji Kim. Terminator: Learning optimal exit points for early stopping in chain-of-thought reasoning. arXiv:2603.12529, 2026.

[38] Ryan Brown, Zihao Fu, and Chris Russell. Settle: Learning when to stop reasoning. arXiv:2609.38997, 2026.

[39] Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Investigating the limitations of transformers with simple arithmetic tasks. arXiv:2102.13019, 2021.

[40] Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. arXiv:2207.04901, 2022.

[41] Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations*, 2022.

[42] Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? a study in length generalization. arXiv:2310.16028, 2023.

[43] Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers can do arithmetic with the right embeddings. In *Neural Information Processing Systems*, 2024.

[44] Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. arXiv:2210.10749, 2022.

[45] Edsger W. Dijkstra and C.S. Scholten. Termination detection for diffusing computations. *Information Processing Letters*, 11(1):1–4, August 1980. ISSN 0020-0190. doi: 10.1016/0020-0190(80)90021-6. URL `http://dx.doi.org/10.1016/0020-0190(80)90021-6`.

A Limitations

Scale. Every experiment runs on one consumer GPU (RTX 5070, 12 GB). To fit a five-hour training budget we cut the pre-registered grid, which called for five seeds per cell and a full maze grid, down to two seeds per method, three single-seed ablations, and a smaller gate-tuning budget for PonderNet and the v1 gate than was pre-registered (`PREREGISTRATION.md`, deviations). With two seeds the accuracy intervals are little more than the seed range, and the pre-registered Wilcoxon tests cannot reach significance. Some comparisons that we report as ties may separate with more seeds, and small differences should not be read as effects.

Mazes. We could not train a maze backbone that passes Gate G1 within the budget; when stopped, it reached 3% exact match on training-size mazes. We therefore make no claims about stopping rules on mazes, and whether local completion heads help when information travels along corridors remains open.

Tasks. The exact theory covers one-dimensional tasks whose determining radius has a closed form. Addition and segmented parity are deliberately simple, so that $\mathrm{LB}(x)$ is known for every instance, and they are not representative of natural-language reasoning. Our conclusions concern recurrent models with a known, local growth rate. They do not transfer directly to models with global attention, whose light cone covers the whole input after a single step.

Backbone dependence. A stopping rule can only be as good as the trajectory it reads. On segmented parity the backbone fails Gate G1, since its stable-oracle accuracy out of distribution is below the pre-registered 95%, so the parity results measure stopping on an imperfect backbone. The progressive loss was also seed-dependent on addition. Termination detection and depth extrapolation are entangled: a rule that looks premature may in fact be reacting to a backbone that never resolves.

What the theory does not say. Theorem 1 rules out *certified* early answers, not correct ones. A rule that halts before $\mathrm{LB}(x)$ can be right on the data distribution, and on addition the convergence detectors often are, because a carry wave keeps the hidden state moving until it lands. This is why we report the premature-stop and premature-error rates of Equation (11) separately, and we do not claim that premature halting is always harmful.

Selection and thresholds. Thresholds are tuned on ID-val by taking the cheapest setting within 0.5 points of the best. A different tolerance would trade accuracy against steps differently. The gate hyperparameters (the ACT ponder weight τ_{ACT}, the PonderNet prior λ_p and the v1 gate weight λ) were tuned on a small grid with one seed, and stronger tuning of the jointly trained gates could narrow some of the gaps we report.

Broader impact. This is basic research on small synthetic tasks. Better termination detection lowers inference compute, and we see no direct path to harm beyond that of adaptive-compute models in general.

B Proofs

Throughout, positions outside the input carry the padding symbol $\perp$, and the latent states and embeddings there take fixed values. Because a window $W_i^{\rho}(x)$ records $\perp$ explicitly, these values are themselves determined by the window.

Proof of Lemma 1. We construct Φ_t by induction on t. For $t = 0$ we have $h_i^{(0)} = H(E(\tilde{x}_i))$, which is a function of $W_i^0(x) = (\tilde{x}_i)$ that does not depend on i; set $\Phi_0 = H \circ E$. Assume the claim holds for t. By Equation (1),

$$h_i^{(t+1)} = U\Big(\big(\Phi_t(W_j^{rt}(x))\big)_{|j-i|\le r},\ \big(E(\tilde{x}_j)\big)_{|j-i|\le r}\Big).$$

For $|j - i| \le r$ the window $W_j^{rt}(x)$ is a contiguous sub-window of $W_i^{r(t+1)}(x)$ at a fixed offset $j - i$, and $\tilde{x}_j$ is an entry of $W_i^r(x) \subseteq W_i^{r(t+1)}(x)$. Since U, E and Φ_t are shared across positions, the right-hand side is a fixed function Φ_{t+1} of $W_i^{r(t+1)}(x)$. For the second statement, W_i^{rs} is the central part of W_i^{rt} when $s \le t$, so equal windows of radius rt give equal states for every $s \le t$, and the pointwise decoder D gives equal predictions. □

Proof of Theorem 1. Let $\rho = rt$. Since $t < \lceil \rho_i(x)/r \rceil$ and t is an integer, $rt < \rho_i(x)$, so by minimality of $\rho_i(x)$ position i is not ρ-determined. Hence there is a pair (x', i') with

$$W_{i'}^{rt}(x') = W_i^{rt}(x) \qquad \text{and} \qquad y_{i'}(x') \neq y_i(x).$$

By Lemma 1, $\hat{y}_{i'}^{(t)}(x') = \hat{y}_i^{(t)}(x)$. A single prediction cannot equal two different targets, so it is wrong for at least one of the two positions. Choosing i with $\rho_i(x) = d^*(x)$ gives the statement about $\mathrm{LB}(x)$. □

Proof of Corollary 1. Suppose position i of x commits at a step $t < \lceil \rho_i(x)/r \rceil$, and take the pair (x', i') supplied by Theorem 1. By Lemma 1 the histories $h_i^{(0)}, \dots, h_i^{(t)}$ on x and $h_{i'}^{(0)}, \dots, h_{i'}^{(t)}$ on x' coincide. Because the done flag is a function of the history alone, position i' of x' raises its flag at the same step t (not earlier, since x would then have raised it earlier as well) and commits the same prediction, which is wrong for one of the two inputs. Hence every position commits no earlier than $\lceil \rho_i(x)/r \rceil$, and since the input halts only when all positions have committed,

$$\tau(x) \;\geq\; \max_i \left\lceil \frac{\rho_i(x)}{r} \right\rceil \;=\; \left\lceil \frac{d^*(x)}{r} \right\rceil \;=\; \mathrm{LB}(x). \qquad \square$$

Proof of Corollary 2. At length L, the input with $a_j + b_j = 9$ for all $j < L$ has no stops, so $d^* = L + 1$ by Proposition 1, and Theorem 1 applies at the final carry position. Any $\tau(L) < \lceil (L+1)/r \rceil$ is therefore wrong on one member of the corresponding pair, and the same argument applies to every input with $d^*(x) > r\,\tau(L)$. Conversely, if $\tau(L) \geq \lceil (L+1)/r \rceil$, the number of wasted steps on x is $\tau(L) - \mathrm{LB}(x) \geq \lceil (L+1)/r \rceil - \mathrm{LB}(x)$. For uniformly random digits each position is a non-stop with probability $1/10$, independently of the others, so $d^* - 1$ is the longest run of successes in L Bernoulli$(1/10)$ trials. By the Erdős–Rényi law for longest runs,

$$d^* - 1 = \log_{10} L + O_p(1),$$

so $\mathrm{LB}(x) = O(\log L)$ with high probability, and the waste is $\Theta(L)$ steps. $\square$

Proof of Proposition 1. Addition. Write c_i for the carry into position i, so that

$$y_i = (a_i + b_i + c_i) \bmod 10 \quad (i < L), \qquad y_L = c_L, \qquad c_{i+1} = \mathbb{1}\big[a_i + b_i + c_i \geq 10\big].$$

If $s = s_i(x) \geq 0$, every position strictly between s and i satisfies $a_j + b_j = 9$ and therefore passes its carry on unchanged, which gives $c_i = c_{s+1} = \mathbb{1}[a_s + b_s \geq 10]$. If there is no stop, all of $0, \dots, i-1$ sum to 9 and $c_i = c_0 = 0$.

Upper bound, $\rho_i \leq i - s_i$. The window of radius $i - s_i$ contains position s_i (or the boundary symbol at -1) and every position in between. It therefore fixes c_i and (a_i, b_i), and hence y_i, for every (x', i') that shares the window. If the window contains $\perp$ at relative offset $-(i - s_i)$, then $i' = i$ in x' as well, and the carry is 0 there too.

Lower bound, $\rho_i \geq i - s_i$. Let $\rho = i - s_i - 1$. The window $W_i^\rho(x)$ covers positions $i-\rho, \dots, i-1$, all of which sum to 9, and contains no boundary symbol on the left. Build x' by copying the window content to centre position $i' = \rho + 1$ and placing at position 0 a single digit pair whose sum is at least 10 if $c_i(x) = 0$ and at most 8 if $c_i(x) = 1$; the right half of the window is copied unchanged. Then $W_{i'}^\rho(x') = W_i^\rho(x)$ but $c_{i'}(x') = 1 - c_i(x)$, so $y_{i'}(x') \neq y_i(x)$.

Segmented parity. The argument is identical, with "delimiter" in place of "stop" and the prefix parity of the current segment in place of the carry. Flipping the bit just outside the window changes the parity. $\square$

Proof of Proposition 2. Addition. Each position stores the class of its pair sum and a carry-in estimate $\hat{c}_i \in \{0, 1, ?\}$, initially ?. At each step, position i reads the carry-out of its left neighbour,

$$\kappa_{i-1} = \begin{cases} \mathbb{1}[a_{i-1} + b_{i-1} \geq 10] & \text{if } i-1 \text{ is a stop,} \\ \hat{c}_{i-1} & \text{if } a_{i-1} + b_{i-1} = 9, \\ 0 & \text{if } i-1 \text{ is the boundary } \perp, \end{cases}$$

and sets $\hat{c}_i \leftarrow \kappa_{i-1}$ if $\hat{c}_i = ?$ and $\kappa_{i-1} \neq ?$. This update reads only radius-1 neighbours and is the same at every position and every length. By induction on $i - s_i$, the estimate $\hat{c}_i$ becomes known at exactly step $i - s_i = \rho_i$: at step 1 for the right neighbour of a stop or for position 0, and one step later for each 9 in between. Once known, it equals the true carry. The done flag is $\mathbb{1}[\hat{c}_i \neq ?]$ and the committed output is $(a_i + b_i + \hat{c}_i) \bmod 10$, which is correct. The input halts at $\max_i \rho_i = d^*(x) = \mathrm{LB}(x)$ for $r = 1$. *Parity* uses the same construction, with the prefix parity of the segment in place of the carry, initialised to 0 at delimiters. $\square$

C Implementation details

Training. We use AdamW with learning rate 10^{-3} and weight decay 10^{-4}, 300 warm-up steps followed by cosine decay, and gradient clipping at 1.0. Training runs in fp32 with TF32 matrix multiplies, which was faster than bf16 autocast for this launch-bound model on our GPU. For T1 and T2 the backbone has width 128 and $r = 1$ and is trained for 8000 iterations at batch size 512 with $T_{\text{train}} = 20$. For T3 it has width 64 and $r = 4$ and is trained for 5000 iterations at batch size 32, with random dihedral augmentation and $T_{\text{train}} = 32$. Training batches are sampled on the GPU and filtered against the ID-val and ID-test hashes. Base models use seeds 0–4 and jointly trained methods seeds 0–2 (T1/T2); mazes use seeds 0–1.

Gate hyperparameters. Every jointly trained gate receives the same budget of three settings on addition (seed 0). Among the settings within 0.5 points of the best ID-val accuracy, the one with the fewest mean ID-val steps is used everywhere (`configs/tune_gates.yaml`). The grids are $\tau_{\text{ACT}} \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ for the ACT ponder weight, $(\beta, \lambda_p) \in \{(0.01, 0.1), (0.01, 0.2), (0.1, 0.05)\}$ for PonderNet, and $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ for the compute weight of the v1 gate.

Post-hoc heads. The post-hoc heads are fitted on the frozen backbone for 1500 iterations (batch 256; batch 32 for mazes) on fresh training-distribution batches rolled out to $2T_{\text{train}}$ steps, with the backbone in bf16 autocast and the heads in fp32. We use Adam with learning rate $2 \cdot 10^{-3}$ and cosine decay. The pooled correctness head (TRM-style) predicts exact match from the masked mean of the latent state, and the completion heads predict the target of Equation (7) per position.

Halting loop. Algorithm 1 spells out the halting rule of Equation (9) as it runs at test time. Instances are processed in batches, and an instance leaves the batch as soon as it halts.

Algorithm 1 Halting with local completion heads (batched; halted instances leave the batch).

Require: input x, backbone (E, H, U, D), completion head c, threshold θ, budget $T_{\max}$

$e \leftarrow E(x)$; $h \leftarrow H(e)$; $p \leftarrow \mathbf{0}$
for $t = 1, \ldots, T_{\max}$ **do**
  $h \leftarrow U(h, e)$; $\hat{y} \leftarrow \arg\max D(h)$
  $p \leftarrow \sigma\big(c(h, p)\big)$ ▷ per position; the pointwise head ignores p
  if $\min_{i \text{ valid}} p_i > \theta$ **or** $t = T_{\max}$ **then**
    return $(\hat{y}, t)$
  end if
end for

Threshold grids. Probability rules use the thresholds

$$\theta \in \{0.5,\ 0.6,\ 0.7,\ 0.8,\ 0.9,\ 0.95,\ 0.98,\ 0.99,\ 0.995,\ 0.999,\ 0.9999\}.$$

Entropy, KL and hidden-state change use 19 log-spaced values in $[10^{-6}, 1]$, stability uses $k \in \{2, 3, 5\}$, ACT thresholds its cumulative halting probability at $\{0.5, 0.7, 0.9, 0.95, 0.99\}$, PonderNet at $\{0.3, 0.5, 0.7, 0.9, 0.95, 0.99\}$, and the fixed-step rule uses $T \in \{1, \ldots, T_{\max}\}$. The test-time budget of each split is

$$T_{\max} = \left\lceil \frac{\max_x d^*(x)}{r} \right\rceil + 16.$$

Metrics. Besides accuracy, mean steps and the premature rates of Equation (11), we record several diagnostics. Let $t^{\text{stab}}(x)$ be the first step from which the answer stays correct, the step chosen by the stable oracle. *Overthinking* is $\tau(x) - t^{\text{stab}}(x)$, averaged over correctly answered instances. *Regret* is the accuracy lost and the extra steps spent relative to the stable oracle, the latter measured on instances that the oracle gets right. *Depth tracking* is the Spearman correlation $\rho_S(\tau, d^*)$, and *length leakage* the partial Spearman correlation $\rho_S(\tau, L \mid d^*)$; a rule that halts at $\text{LB}(x)$ depends on depth alone and therefore leaks no length. Calibration is the 15-bin expected calibration error of the per-position done probabilities.

Compute. Table 5 lists the measured GPU time of each sweep.

Table 5: Measured GPU time per sweep (RTX 5070; several runs shared the GPU, so wall-clock per run is higher than in isolation). Runs listed in more than one sweep are counted once per sweep.

Sweep	Runs	Train (h)	Eval (h)
`ablations`	3	0.6	0.2
`main_addition`	8	2.6	0.6
`main_gates`	12	5.2	1.3
`main_parity`	8	2.4	1.3
`pilot_addition`	8	2.4	1.1
`tune_gates`	2	1.1	0.2
Total	41	14.3	4.8

D Additional results

The DepthShift splits. Figure 2 plots the length and depth of every frozen test instance. Because OOD-depth and OOD-length each differ from ID in a single variable, the effect of one can be measured with the other held fixed.

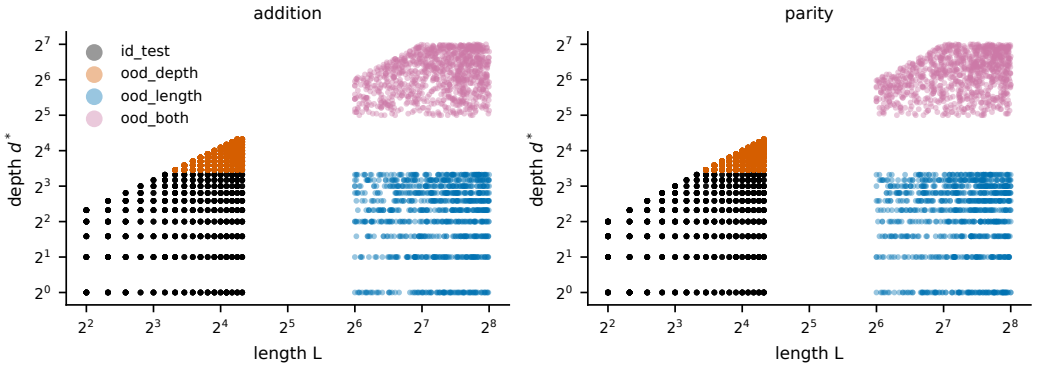


Figure 2: Length and depth of every frozen test instance. OOD-depth and OOD-length each differ from ID in a single variable, so the effects of length and of depth can be measured separately.

The v1 model, re-evaluated. The original GLWM paper reported a mean of 1.05 steps with a gate initialised to halt early. Re-running it with the evaluation of this paper (deduplicated test set, exact match over all positions, Table 6) shows that both initialisations reach full accuracy, the v1 initialisation uses 2.6 steps on average, and only 84% of instances are already correct after one step (Figure 3). The earlier step count came from a halting criterion that stopped before the answer had settled.

Table 6: The v1 GLWM model with a fixed gate initialisation versus the original (v1) initialisation, evaluated on the deduplicated 4–8 digit test set.

Gate init	Accuracy	Mean steps	Steps (4-digit)	Acc. at step 1	MFLOPs/sample
fixed	1.0000 ± 0.0000	3.41	3.87	0.5888	7.50
v1	1.0000 ± 0.0000	2.60	3.05	0.8405	7.02

Per-step accuracy of the backbones. Figure 4 shows exact-match accuracy at every step for both objectives. On addition, accuracy on every split rises to its plateau and stays

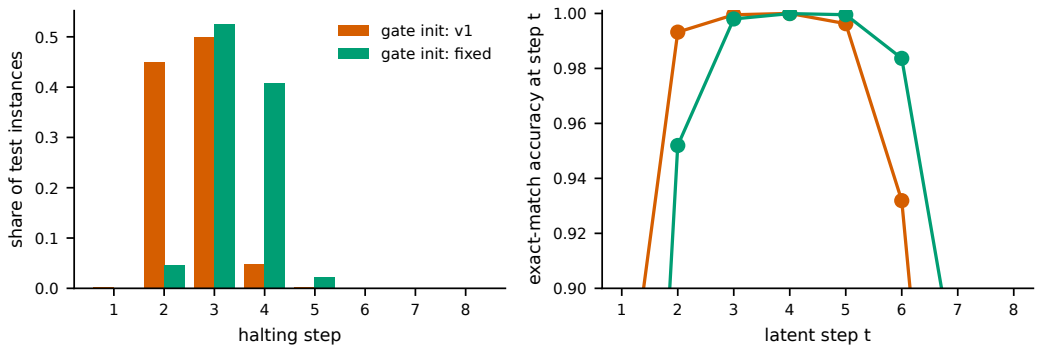


Figure 3: v1 GLWM re-evaluated: halting steps (left) and exact-match accuracy at each latent step (right) for the two gate initialisations.

there: once an answer is right it stays right. On parity, OOD-both accuracy peaks and then decays for both objectives, so the stable oracle sits below 100% and the backbone fails Gate G1.

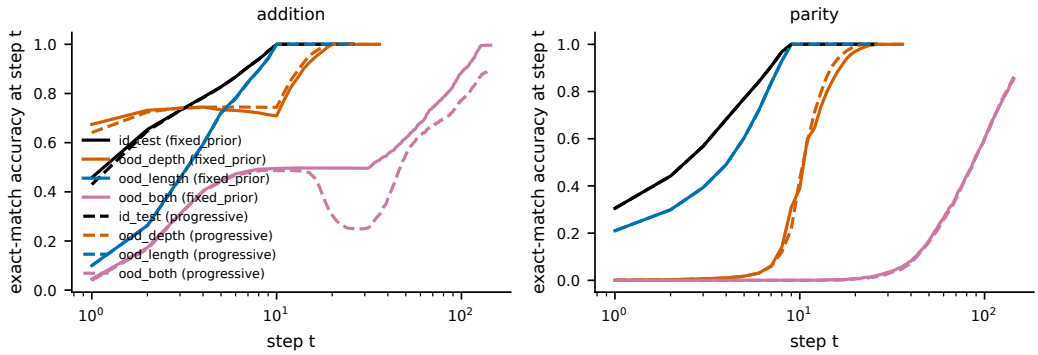


Figure 4: Exact-match accuracy at step t (mean over pilot seeds), fixed-prior (solid) and progressive (dashed) objectives.

Information travels at the light-cone speed. Figure 6 shows, for one OOD-both addition instance, which output positions are correct at which step, together with the per-position lower bound ρ_i from Proposition 1. No position can be certified before its bound, and the backbone's positions settle along a wave that follows it, travelling one position per step from each stop. Figure 7 shows the done probabilities of the two post-hoc completion heads on a deep instance: they switch on along the same wave, after the bound.

Linear probes. A linear probe fitted on ID-val hidden states decodes whether a position is still pending ($t < \rho_i$) from $h_i^{(t)}$, and transfers to OOD-depth and OOD-both without refitting (Table 7). The backbone therefore represents the pending bit of the automaton in Proposition 2; whether a stopping rule uses it is a separate question.

Measured compute. Figure 10 plots measured FLOPs per instance (from `FlopCounterMode`, at the mean length of each split) against accuracy. The completion heads add one small MLP per position and step; their cost is dominated by the number of steps they choose.

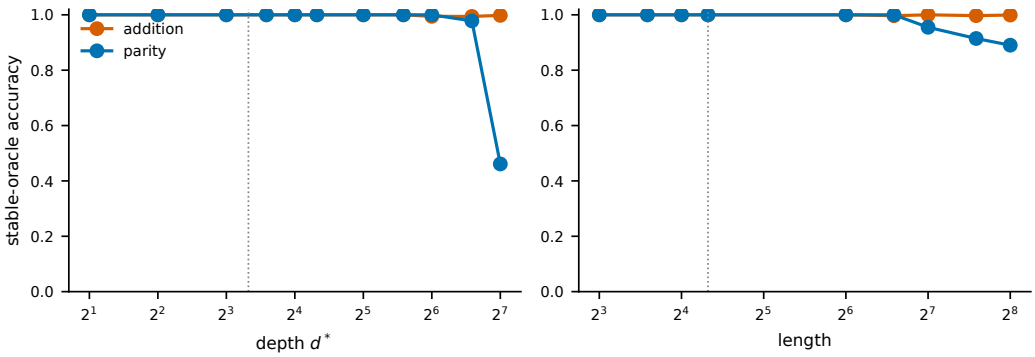


Figure 5: Stable-oracle accuracy against depth (left) and length (right), pooled over splits; dotted lines mark the training maxima. Length alone costs nothing; accuracy on parity drops with depth.

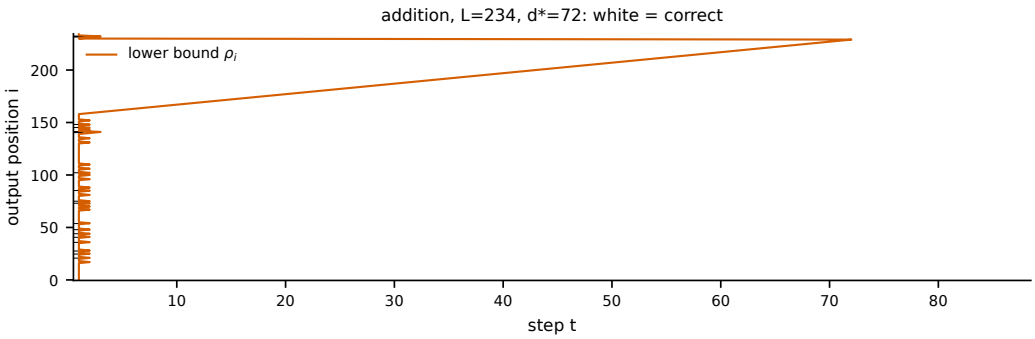


Figure 6: Correctness of every output position (rows) at every step (columns) for one OOD-both addition instance; white is correct. The orange line is the per-position lower bound ρ_i.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state each claim with the experiment that tests it; `paper/claims_map.md` maps every number to its source file.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

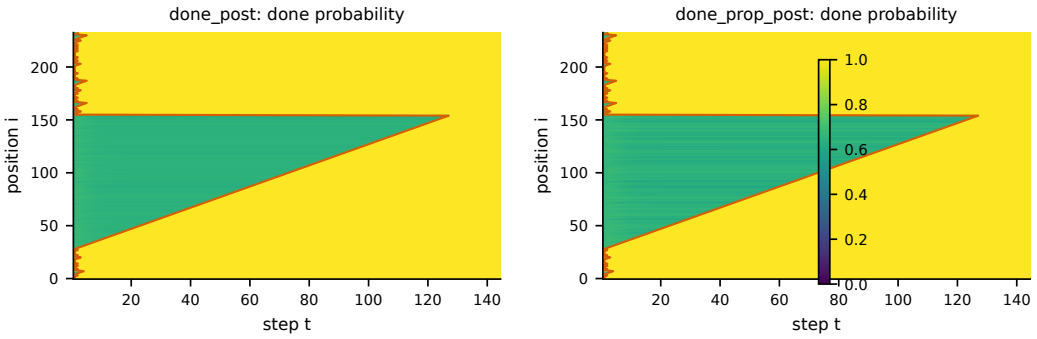


Figure 7: Done probabilities of the post-hoc completion heads (pointwise, left; propagating, right) on a deep OOD-both addition instance, with the per-position lower bound in orange.

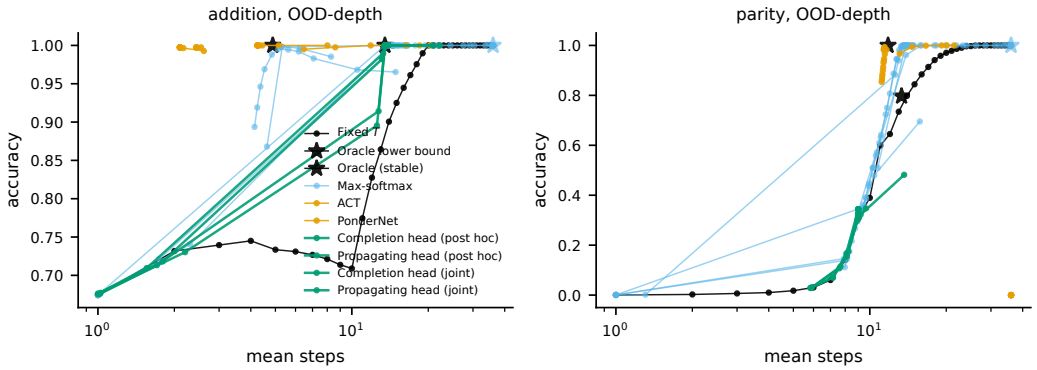


Figure 8: Accuracy against mean steps on OOD-depth for every threshold of every rule.

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section A discusses the tasks, the architecture class, the reduced scale, the maze negative result, and the cases where simple confidence rules suffice.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text

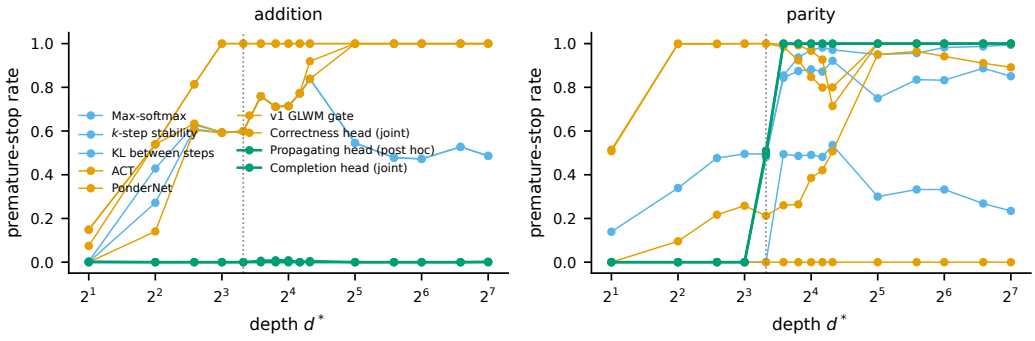


Figure 9: Premature-stop rate against depth; the dotted line is the training maximum.

Table 7: Left: linear probe for the pending state (fitted on ID-val, seed-0 base model on addition). Middle: ablations on addition (seed 0; accuracy in %). Right: CUDA-event latency for 512 OOD-depth instances with online early exit (median ms).

Split	Acc.	AUC
id-test	100.0	1.000
ood-depth	100.0	1.000
ood-both	100.0	1.000

Ablation	Rule	OOD-depth	OOD-both
main	max softmax	100.0	99.8
main	oracle stable	100.0	99.6
no recall	max softmax	100.0	7.5
no recall	oracle stable	100.0	0.6
r 2	max softmax	100.0	100.0
r 2	oracle stable	100.0	100.0
width 64	max softmax	100.0	48.0
width 64	oracle stable	100.0	95.2

Rule	ms
fixed T (val-selected)	7.4
fixed T = T max	23.8
max softmax	19.8
done post	23.8
done prop post	28.3

system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.

- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Assumptions are stated in Section 2; full proofs are in Section B, and each result has an executable check in `tests/test_theory.py`.

   Guidelines:

Table 8: Closest works by property: per-instance depth (PD), depth/length separation (DL), halting tested deeper than training (OOD-H), local halting signal (Loc), formal stopping result (Th), halting signal decoupled from the task-loss weighting (Dec). Y yes, ∼ partial, – no. Full matrix and search protocol: `paper/related_work_matrix.md`.

Work	PD	DL	OOD-H	Loc	Th	Dec
ACT [5]	–	–	–	∼	–	–
PonderNet [6]	–	–	∼	–	–	–
Neural GPU limits [12]	∼	∼	–	–	–	–
DT-recall [2]	–	–	–	–	–	–
AdRNN [19]	∼	–	Y	–	–	–
Looped TF [8]	∼	–	∼	–	–	–
Recurrent depth [3]	–	–	–	Y	–	Y
TRM [4]	–	–	–	–	–	∼
ANIRA [20]	∼	–	∼	–	–	–
Stochastic stopping [26]	–	∼	∼	–	–	–
Halting diagnosis [27]	–	–	–	–	–	Y
Finish detection [28]	–	–	∼	–	–	Y
This work	Y	Y	Y	Y	Y	Y

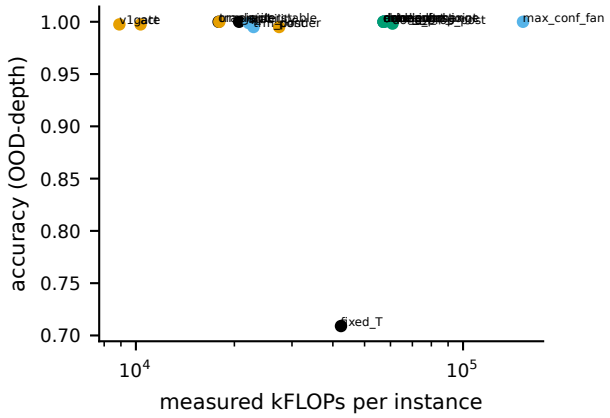


Figure 10: Accuracy on addition OOD-depth against measured kFLOPs per instance for every stopping rule.

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

Justification: Section C lists every hyperparameter, split and selection rule; one YAML config per run and `REPRODUCE.md` give the exact commands.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: All code, configs, frozen-split hashes and the analysis notebooks are released; the data are generated from fixed seeds.

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 3 and 4 and Section C give splits, objectives, optimiser, schedules, threshold grids and the ID-val selection rule.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Bootstrap intervals over seeds (accuracy) and over instances (premature rates); with only two seeds the intervals are close to the seed range and Wilcoxon/Holm tests cannot reach significance, which Sections A and 5 state explicitly.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
   - If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: One RTX 5070; GPU-hours per sweep are reported in Section C.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses synthetic data only and follows the NeurIPS Code of Ethics.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

    Justification: Foundational research on halting in small synthetic-task models; we see no direct societal impact beyond general efficiency of iterative models.

    Guidelines:

    - The answer NA means that there is no societal impact of the work performed.
    - If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: No pretrained language or image models or scraped data are released.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: PyTorch (BSD) and the arXiv metadata used for the bibliography are credited; maze data are generated by our own code.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The DepthShift generators and stopping-rule library are documented in the repository README and `REPRODUCE.md`.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: No crowdsourcing or human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: No human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: An AI coding assistant was used to write code and draft text under the author's direction; all results come from the released code and were checked by the author. LLMs are not part of the method.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.